



Explorer



calculate...

```
1 m=float(input())
2 v=float(input())
3 p=m*v
4 print(f"{p:.2f}kgm/s")
```

```
1 n=int(input(""))
2 v if 1<= n <=9:
3   —> print(n**2)
4 v elif 10<= n <=99:
5   —> print(f"{n**(1/2):.2f}")
6 v elif 100<= n <=999:
7   —> print(f"{n**(1/3):.2f}")
8 v else:
9   —> print("Invalid")
```

```
1 # from datetime import date
2 from datetime import datetime
3
4 d1 = datetime.strptime(input(), "%Y-%m-%d")
5
6 d2 = datetime.strptime(input(), "%Y-%m-%d")
7
8 print((d2 - d1).days)
```

 reverseN...

```
1 n=int(input())
2 reversed_n=0
3 while n > 0:
4     digital = n%10
5     reversed_n = reversed_n * 10 + digital
6     n //= 10
7 print(reversed_n)
```



```
1 n = int(input())
2 for i in range(1,11):
3     print(n, "x", i, "=", n*i)
```

```
def fib(n):  
    → if n <= 0:  
    → → return 0  
    → elif n == 1:  
    → → return 1  
    → else:  
    → → return fib(n-1) + fib(n-2)  
    → →  
n=int(input(.))  
for i in range(n):  
    ..... print(fib(i),end=" ")  
▲
```



```
1 rows = int(input())
2 troublemaker = '*'
3 for i in range(1, rows+1):
4     print(troublemaker*i)
```



numpyarr...

```
1 import numpy as np
2 row, cols = map(int, input().split())
3 elements = []
4 for i in range(row):
5     →elements.extend(map(int, input().split()))
6 array= np.array(elements).reshape(row,cols)
7 print(array)
8 print(array.ndim)
9 print(array.shape)
10 print(array.size)
```

Input matrices

print("Enter Matrix A:")

matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")

matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

Addition

print("Addition (A + B):")

print(matrix_a + matrix_b)

Subtraction

print("Subtraction (A - B):")

print(matrix_a - matrix_b)

Multiplication (element-wise)

print("Element-wise Multiplication (A * B):")

print(matrix_a * matrix_b)

Matrix multiplication (dot product)

print("A dot B:")

print(np.dot(matrix_a, matrix_b))

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
print('Horizontal Stack:')
print(np.hstack((arr1, arr2)))

# Perform vertical stacking (vstack)
print('Vertical Stack:')
print(np.vstack((arr1, arr2)))
```

```
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split()))))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
search_result = np.where(array1==search_value)
print(search_result[0])

# Count occurrences in array1
count_result = np.count_nonzero(array1==count_value)
print(count_result)

# Broadcasting addition
broadcast_result = array1 + broadcast_value
print(broadcast_result)

# Sort the first array
sort_result = np.sort(array1)
print(sort_result)
```

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even
in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

```
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=
["Name", "Age", "Grade"])
# write your code here.
df=pd.DataFrame(data)
print("First five rows:")
print(df.head(5))
print("Average age:",df["Age"].mean().round(2))
print("Students with a grade up to B")
filtered_df=df[df['Grade'].isin(['A', 'B'])]
print(filtered_df)
```

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df['Date'])

df['Total_sales'] = df['Quantity'] * df['Price']

df['Month'] = df['Date'].dt.to_period('M')

# Find the month with the highest total sales
monthly_sales = df.groupby('Month')['Total_sales'].sum()
best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()

print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
product_sales = df.groupby('Product')['Quantity'].sum()

best_product = product_sales.idxmax()

total_quantity = product_sales.max()
# Find the product with the highest total quantity sold
best_product = product_sales.idxmax()
highest_quantity = product_sales.max()

# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# 1. Display the first 5 rows of the dataset
print(data.head(5))

# 2. Display the last 5 rows of the dataset
print(data.tail(5))

# 3. Get the shape of the dataset
print(data.shape)

# 4. Get a summary of the dataset (info)
print(data.info())

# 5. Get basic statistics of the dataset
```

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# 1. Display the first 5 rows of the dataset
print(data.head(5))

# 2. Display the last 5 rows of the dataset
print(data.tail(5))

# 3. Get the shape of the dataset
print(data.shape)

# 4. Get a summary of the dataset (info)
print(data.info())

# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values
print(data.isnull().sum())
```

```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June', 'July',
    'August', 'September', 'October', 'November', 'December'],
    'City_A_Temperature': [5, 7, 10, 13, 17, 20, 22, 21, 18, 12, 8, 6],
    'City_B_Temperature': [2, 3, 5, 6, 10, 14, 16, 17, 12, 9, 5, 3],
    'City_C_Temperature': [3, 4, 6, 8, 9, 12, 15, 14, 10, 7, 4, 2]
}
Months=data['Month']
CityA=data['City_A_Temperature']
CityB=data['City_B_Temperature']
CityC=data['City_C_Temperature']
plt.stackplot(Months, CityA, CityB, CityC)
plt.title("Temperature Variation")
plt.ylabel("Temperature")
plt.xlabel("Month")
plt.show()
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Histogram
plt.hist(data['Age'], bins=30, edgecolor='k')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Age Distribution')
plt.show()
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Gender
survival_counts = data.groupby('Sex')['Survived'].value_counts().unstack()
survival_counts.plot(kind='bar', stacked=True)

plt.title('Survival by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.legend(['Not Survived', 'Survived'])
plt.show()
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Age by Pclass
data.boxplot(column='Age', by='Pclass')
plt.title('Age by Pclass')
plt.suptitle('')
plt.xlabel('Pclass')
plt.ylabel('Age')
plt.show()
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Age by Survived

plt.figure(figsize=(8,6))
data.boxplot(column='Age',by='Survived')
plt.title('Age by Survival')
plt.suptitle('')
plt.xlabel('Survived')
plt.ylabel('Age')
plt.show()
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Scatter Plot for Age vs. Fare by Survived

colors = data['Survived'].map({0: 'red', 1: 'blue'})

plt.scatter(data['Age'], data['Fare'], c=colors)
plt.title('Age vs. Fare by Survival')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```